PRODUCT REQUIREMENTS DOCUMENT

Code Review Workbench

Version 0.3

11 October 2026

Help a developer understand a code change well enough to make their own review decision.

PRODUCT	Reviewer-controlled, agentic VS Code workbench
STATUS	Shared product understanding confirmed
SCOPE	Product behavior and functional requirements
FORMAT	Illustrated monochrome booklet / 24 pages

This edition replaces draft v0.2 where alignment decisions change it. Provider compatibility, practical limits and detailed interface treatment remain open. Architecture, implementation and scheduling belong to later work.

Contents and reading guide

Section	Page
01. Product definition and users	3
02. MVP scope and entry modes	4
03. PR selection and local lifecycle	5
04. Workspace and navigation	6
05. Review journey and orientation	7
06. Context and evidence	8
07. Synchronization and continuity	9
08. Suggestions and coverage	10
09. GitHub discussion and feedback	11
10. Review decisions and merging	12
11. Verification and command interaction	13
12. Profile, AI access and private data	14
13. FR01-FR04: opening and orientation	15
14. FR05-FR08: conversation and comments	16
15. FR09-FR12: suggestions, reviews and merges	17
16. FR13-FR16: context and verification	18
17. FR17-FR20: practice and workspace	19
18. Open questions and future direction	20
19. General market comparison	21
20. References and revision basis	22
21. Theoretical strengths	23
22. Workflow friction and tradeoffs	24

How to read this booklet. Pages 3-14 describe the product experience. Pages 15-19 contain the normative functional requirements, preserving FR01-FR20 for continuity. Pages 20-24 distinguish open questions, external evidence and theoretical positioning.

Requirement language. "Must" identifies agreed behavior. "Deferred" identifies work outside MVP. "Open" identifies an unsettled detail or unproven capability. Observable checks concern whether the product behaves as specified; they do not score the reviewer or prescribe review depth.

01 / Product definition and users

Product statement

The Code Review Workbench is a VS Code extension that connects code inspection, private AI investigation, existing PR discussion, optional verification and GitHub collaboration within a continuing review context. The assistant helps the developer understand the change. The developer decides what to inspect, what matters, what to publish and which review decision to make.

The product's intended value is less repeated reconstruction of context while moving among a diff, code, assistant conversation, terminal and GitHub discussion. That is an intended benefit, not a demonstrated productivity claim.

Who it serves

User or participant	Role in the product
Peer reviewer	Primary journey: understand another developer's change and provide chosen feedback.
Author reviewing their own work	Inspect local or published changes using the same understanding tools. GitHub still governs available review actions.
GitHub collaborator	Continue comments and replies entirely on GitHub without adopting the workbench.

Initial exploration emphasizes multi-file and AI-assisted changes, while supporting human-written changes. AI authorship is not treated as evidence that code is defective. Entry is language agnostic; mixed-language PRs are explicitly part of MVP.

Product principles

- **Understanding belongs to the human.** The initial assessment and suggestions are provisional assistance, never automatic approval.
- **Code stays central.** The workbench preserves ordinary editor use and keeps explanations close to source.
- **Review is flexible.** Suggested checkpoints, suggestions and focused checks are optional.
- **Evidence remains attributable.** Source observations, collaborator statements, hypotheses and command outcomes retain their distinct meaning.
- **Publication is deliberate.** Private conversation and unselected AI feedback do not become GitHub comments automatically.

Review autonomy

The workbench does not judge whether a review was sufficiently thorough, quick or good. The reviewer chooses how much time and attention to invest. There are no formal review-success metrics, review-quality scores or repeat-use targets in this PRD. The product must still report factual operational outcomes, including whether a command stopped, publication is uncertain or GitHub confirms a merge.

02 / MVP scope and entry modes

Supported review modes

Mode	Included behavior	Collaboration destination
GitHub PR review	Accessible public and authorized private GitHub.com PRs; peer and self-review; published discussion.	Associated GitHub PR, subject to permission and valid source correspondence.
Local committed comparison	Compare two commits or branches, such as a feature branch against main.	Private local investigation unless a valid PR destination is associated.
Local uncommitted review	Inspect working changes before a PR exists.	Private local investigation.

MVP supports one active review at a time. Other PRs remain in the personal list and retain saved progress. Repository-level synchronization remains a batch operation across relevant PRs; it does not imply several simultaneous active reviews.

Included feature groups

Group	MVP commitment
Entry and orientation	Relevant-PR selection, PR links, local setup, short briefing, optional route and file orientation.
Investigation	Continuing same-PR context, related-code reads/search, inspectable sources and text attachments.
Assistance	Reviewer-paced suggestions, category/impact labels, deduplication, dismissals and visible examination scope.
Collaboration	GitHub discussion, comments/replies, selected feedback, pending reviews, explicit submission and permitted merging/queueing.
Verification and practice	Optional terminal checks, normal permissions, local state, one editable profile and supported cloud AI access.

Outside MVP

Dedicated PR source-editing features, local models and third-party executable skills/add-ons are deferred for now. Other Git hosts, a standalone editor, public marketplaces, organization administration, exhaustive repository audits and sensitive file/folder exclusion controls are outside MVP. Automatic cross-device profile sync, scanned-PDF interpretation and proactive cross-PR review memory are not included.

Ordinary VS Code editing and task-scoped terminal commands remain available. The dedicated-editing boundary does not make the editor read-only and does not add a separate source-change approval layer. Autonomous fixing is not an agreed workflow.

Language boundary: Any-language and mixed-language entry is required. Equal navigation support, verification tooling or AI quality in every language is not promised.

03 / PR selection and local lifecycle

Personal selection model

The PR list starts with a top-level repository selector and then shows that repository's relevant PRs. Relevance includes mentions, assignments, requested reviews, the user's authored PRs, manually added PR links and PRs already reviewed. Open PRs are shown by default; a status filter exposes relevant closed or merged PRs. There is no default feed containing every PR or vertically stacked repository groups.

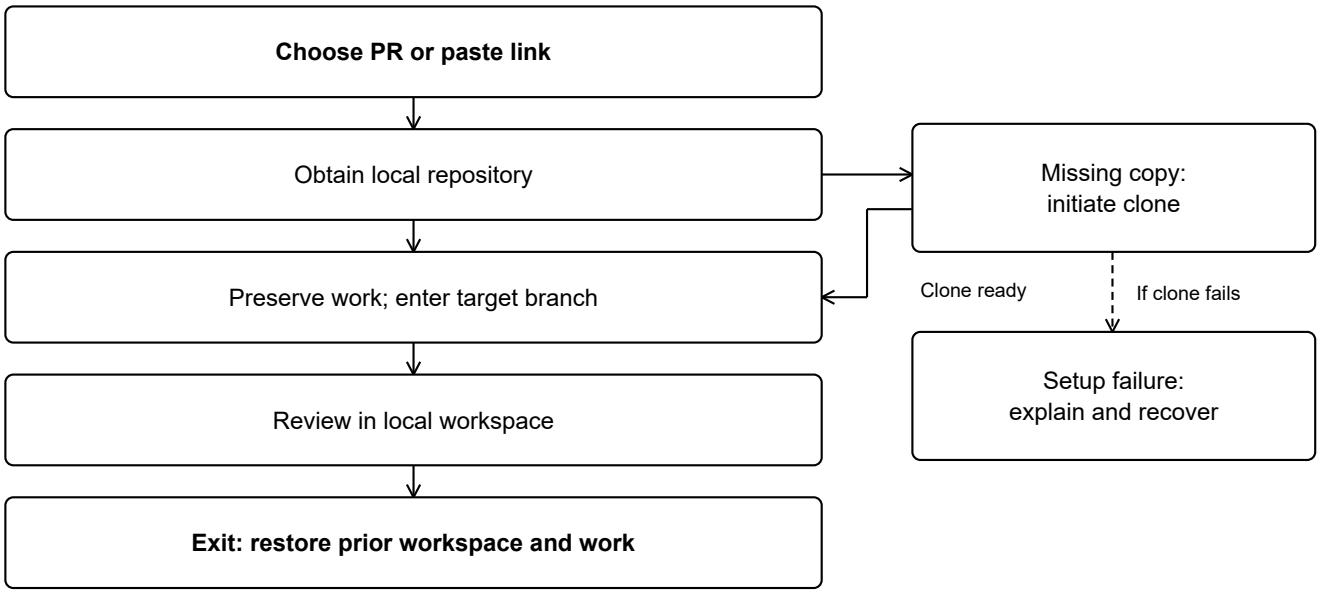


Figure 01. Local review lifecycle. Preservation and recovery are functional obligations; no Git mechanism is prescribed.

Preservation and failure behavior

- Entering a PR review automatically opens the target repository and checks out the PR branch, while preserving pre-review unfinished work.

- A local repository is mandatory. If cloning fails, explain the failure and offer recovery; do not claim a remote-only review has started.

- When the repository is already local but prior work cannot be safely preserved, allow reading without switching branches and leave that work intact.

- Explicit Exit Review restores the previous workspace, repository, branch and unfinished work. Collapsing the assistant leaves the review active.

- Clone, checkout, source-access and restoration failures require actionable recovery. The product must not present a failed transition as completed.

04 / Workspace and navigation

The extension uses familiar VS Code surfaces. Source references open the relevant diff, file or surrounding code in ordinary editor tabs. Supported definition/reference navigation is reused, with existing workspace search available as a fallback. Code remains the main central surface.

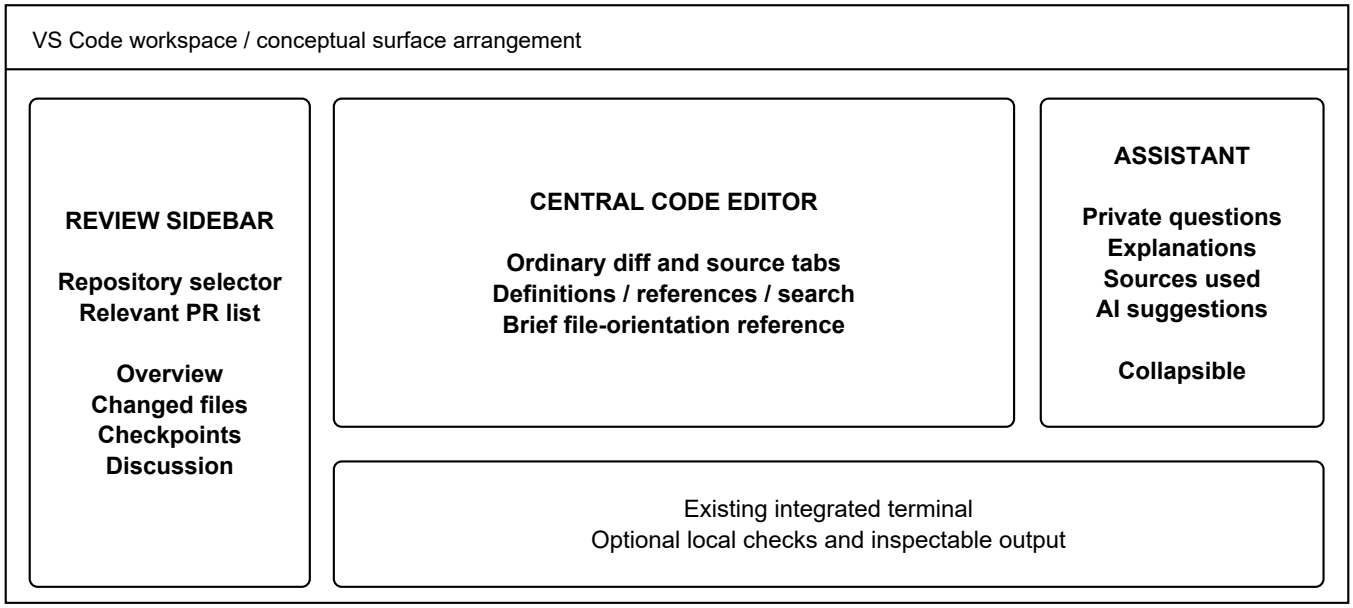


Figure 02. Conceptual workspace allocation. It establishes surface relationships, not fixed dimensions or a finished interface design.

Surface responsibilities

Surface	Product responsibility
Review navigation	Access overview, changed files, optional checkpoints, discussion and suggestions. Exact menu names remain design details.
Code editor	Read changed and surrounding code; follow source references; keep ordinary tabs and editor behavior.
Assistant	Explain orientation, investigate questions, show incremental suggestions and inspectable tool activity.
Terminal	Use local tools for reviewer-chosen focused verification.

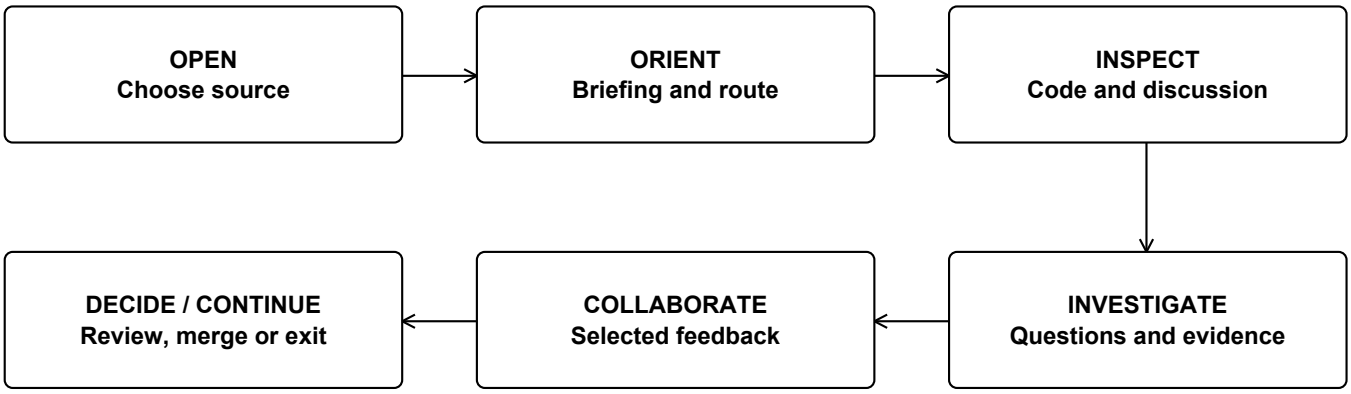
Interaction requirements

The assistant can be collapsed. Narrow-window adaptation must keep code usable and central. Review workflows must be keyboard accessible and respect the editor theme. Private drafts, AI suggestions and GitHub-published content require explicit text labels; color alone must not communicate their status.

Opening code or reading an explanation does not mark a file Reviewed. The reviewer sets human Reviewed marks explicitly. AI examination status is a separate account of the areas the agent has inspected.

The booklet's black-only styling applies to this artifact. It does not change the agreed requirement for the product to follow the user's editor theme.

05 / Review journey and orientation



Direct navigation remains available. Routes, suggestions and checks are optional.

Figure 03. The review journey is iterative. The reviewer can navigate directly, skip assistance and return after updates.

Stage	Reviewer experience	Product behavior
Open	Choose a PR/link or local comparison.	Establish local source, review identity and available discussion.
Orient	Read the brief opening explanation.	Synthesize description and code, flag disparities and give a provisional take.
Inspect	Follow file order or optional checkpoints.	Keep all changed files available and explain a file's role in the PR/route.
Investigate	Ask privately, inspect a suggestion or choose a check.	Gather relevant supporting code and expose evidence/limits.
Collaborate	Compose feedback or reply.	Publish only the reviewer-selected final content under their identity.
Decide	Choose Comment, Approve or Request changes.	Submit the chosen review; handle source updates accurately.
Continue or exit	Merge if permitted, resume later or leave.	Report GitHub's actual state; preserve progress and restore prior work on exit.

Briefing and checkpoints

The opening summary explains intended behavior and the change in plain language. It considers both implementation and PR description; material disparities are mentioned without separate "author intent" and "inferred intent" labels. Missing requirements must not be invented.

The suggested route groups meaningful areas, such as permission rules, request handling and related tests. The reviewer can ignore it, reorder it through an agent request, or drag checkpoints with equivalent keyboard controls. Every changed file stays accessible.

Opening code for review yields a brief orientation summary explaining its role in the overall PR and current checkpoint. The reviewer and agent can use that summary in follow-up questions. This is balanced file-level orientation, not tracking every cursor movement as primary context.

06 / Context and evidence

Continuing review context

The primary context is the current PR's changed code and conversation so far. Relevant guidance and supporting sources supplement it. The agent reads/searches related code as needed; it does not load the entire repository into every interaction or require a manual retained-source picker.

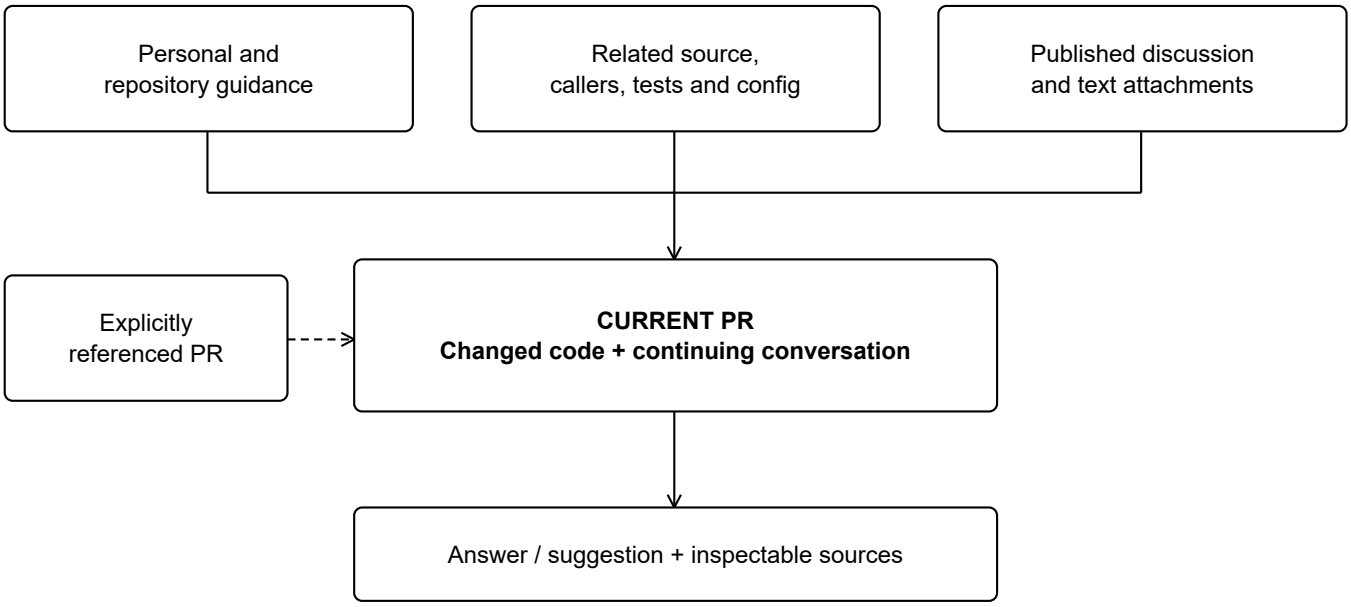


Figure 04. Logical context model. Dashed access to another PR requires an explicit reference; it does not start another active review.

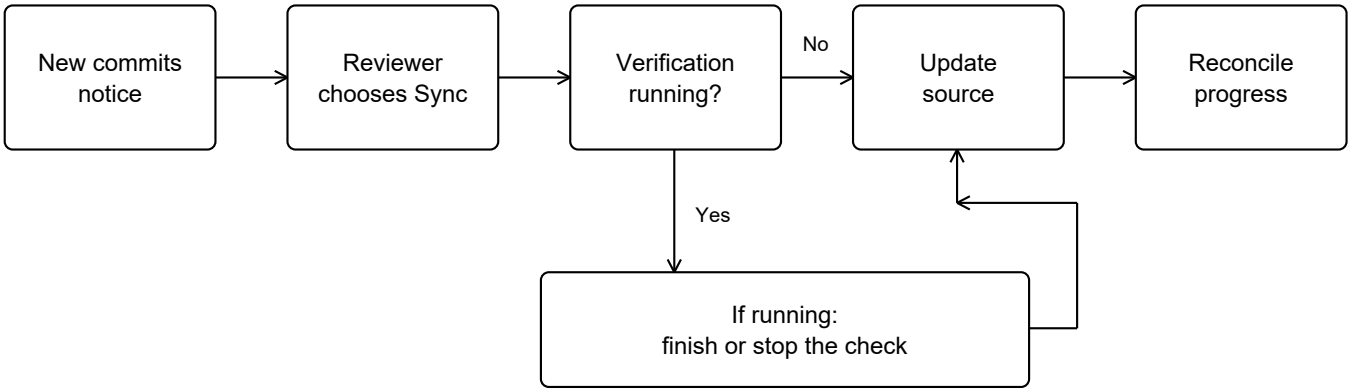
Evidence distinctions

Material	Required interpretation
Source observation	Describes the code actually examined, with a relevant reference.
Collaborator statement	Attributed discussion, not automatically a verified implementation fact.
Hypothesis or concern	An inference whose reasoning and limitations remain clear.
Human decision	An agreed same-PR tradeoff or correction, retained as a decision.
Command output	What a real command produced; bounded by actual execution and known uncertainty.

Sources used to produce an answer must be inspectable. Exact presentation remains open. Users contribute or correct context through chat and attachments: plain text, source code, Markdown and PDF text. Image-only/scanned PDF interpretation is outside MVP. Missing or partially readable sources must not be represented as fully understood.

Same-PR decisions remain in context. Other PR context may be retrieved when explicitly referenced in the description or other relevant material. The agent must not proactively discover relationships or inject unprompted decisions from earlier reviews. Shared repository and personal guidance remains available.

07 / Synchronization and continuity



Keep unaffected state. Recheck affected or uncertain evidence. Discussion refresh is separate.

Figure 05. Active-review source synchronization. Discussion refresh is independent of this source transition.

Event or action	Required behavior
Open/resume PR	Automatically synchronize latest source and restore same-PR progress.
New commits during active review	Show a visible notice; checked-out source updates when the reviewer chooses Sync now.
Per-review Sync	Update the current PR's source and review context.
Repository/list Sync	Apply review Sync across all personally relevant PRs in the selected repository, including an active PR when relevant. Initiate clone if the repository is missing locally.
Published discussion changes	Refresh new comments/replies independently of source Sync.

Reconciliation rules

Clear the current human Reviewed mark for a changed file; preserve marks on unchanged files. Keep unaffected drafts, dismissals and context. Flag affected evidence as outdated or needing rechecking, including uncertain dependencies. An older observation, review approval or check result never becomes current verification simply because it was retained.

Keep existing route checkpoints when incoming changes fit them. Reflect a newly introduced area when the old route does not cover it. Do not rebuild the plan unnecessarily after every update.

Source synchronization of the active PR waits for a running workbench verification task to finish or be stopped. The reviewer can stop that work to proceed. Preservation and recovery rules still apply to both individual and repository-level Sync.

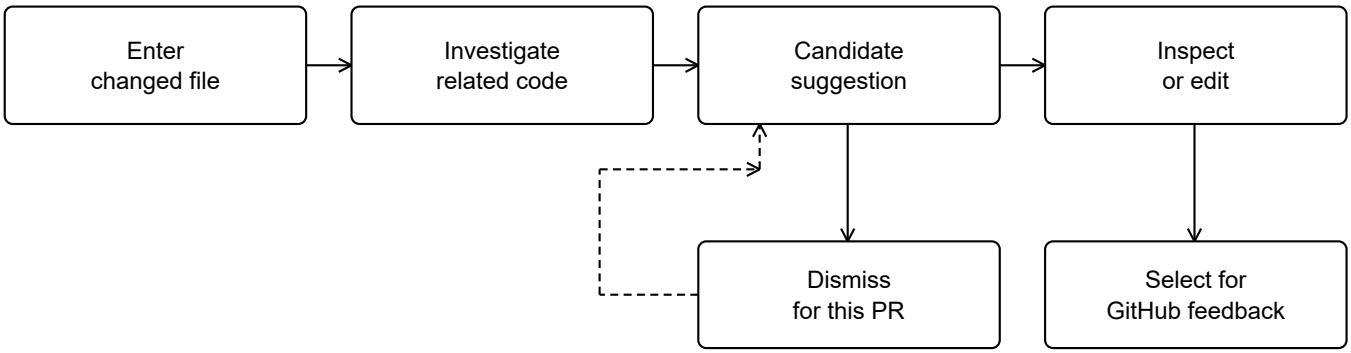
08 / Suggestions and coverage

Labels and compact presentation

Category	Meaning
Bug / Security	Correctness concern / security concern.
Performance / Maintainability	Efficiency concern / maintainability concern.
Tests / Documentation / Nit	Test concern / documentation concern / optional polish.

Impact is separate: **Critical** means severe harm; **Major**, substantial harm; **Minor**, limited impact; **Trivial**, polish. The reviewer can adjust impact. These are agreed product labels, not a claim that most developers prefer them. Evidence certainty and merge-blocking intent are not implied by a category or impact tag.

Use simple tags and explanatory text, with the explanation collapsed by default. On expansion, explain the concern, why it matters, applicable location, inspected sources and verification limits. Useful code and discussion links sit inside the text; a separate title/location card is not required.



A return after dismissal requires meaningful new evidence or relevant changes.

Figure 06. Suggestions follow reviewer pace. A dismissal resurfaces only after meaningful new evidence or relevant code changes.

Pace, coverage and noise

Generate the next increment as the reviewer enters a changed file, growing relevant context gradually. The opening PR briefing does not imply an initial whole-PR suggestion pass. Related-code investigation is still permitted.

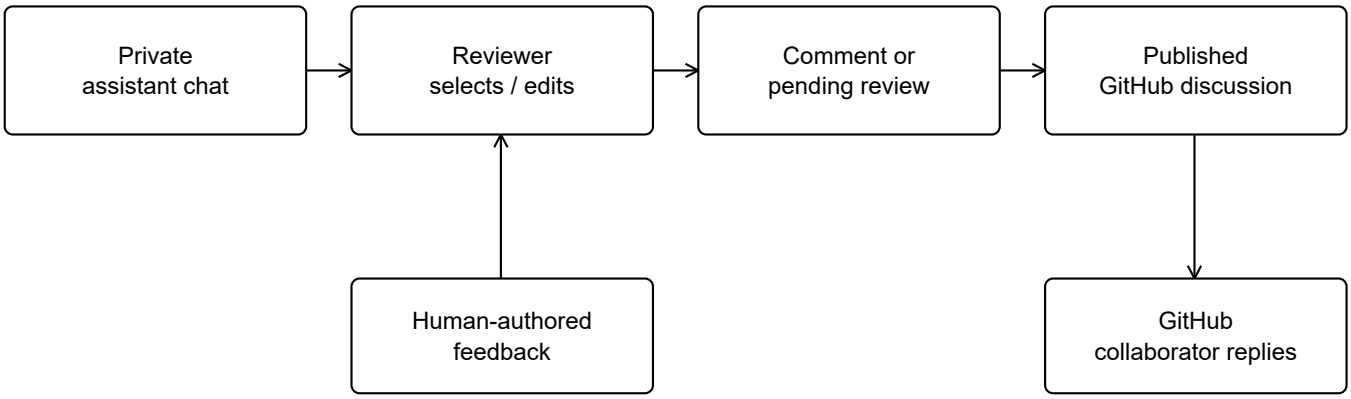
Show agent-examined and unexamined areas separately from explicit human Reviewed marks. An unexamined file with no suggestions has not been cleared. The reviewer is not required to inspect or resolve every suggestion.

Combine overlapping suggestions about one underlying issue while retaining relevant locations. Link existing current-PR discussion rather than repeat its concern. Dismissal is one action with an optional reason and persists for that PR across resumes. Suggestions remain private candidates until the reviewer deliberately adopts and publishes selected final text.

09 / GitHub discussion and feedback

Conversation in the workbench

Import accessible published PR-level comments, submitted review summaries and inline threads/replies, with attribution, source anchors and links. Preserve available resolved/outdated states. Fetch the accessible conversation rather than presenting a partial load as complete. Automatically refresh new published discussion while active; show known loading failures or incomplete scope.



Unselected chat and suggestions remain private from GitHub collaborators.

Figure 07. Private understanding becomes collaboration only through the reviewer's selected feedback. Teammates can remain on GitHub.

Feedback destinations and control

Capability	Required boundary
PR-wide, file or line/range feedback	Use an accurate location accepted by GitHub. Older-code feedback may retain its valid earlier anchor.
Human questions and adopted AI suggestions	Publish the visible final text under the signed-in reviewer's GitHub identity.
Exact replacement suggestion	Use GitHub suggested-change feedback for the author to apply; no dedicated local fixing workflow is implied.
Reply / edit / delete / resolve / reopen	Stay within the original conversation and the operator's GitHub permissions; edit/delete applies to their own comments.

Assistant chat is private from GitHub collaborators by default. It and unselected AI suggestions are not silently included in a review. Cloud model processing is a separate data boundary described on page 14.

Follow GitHub's state-dependent flow: **Comment** for immediate individual feedback; **Start a review** for a pending batch; **Add review comment** for an existing pending review. Restore the reviewer's own pending review and unfinished feedback on return. Permit discarding the pending review without publishing. S02, S03

10 / Review decisions and merging

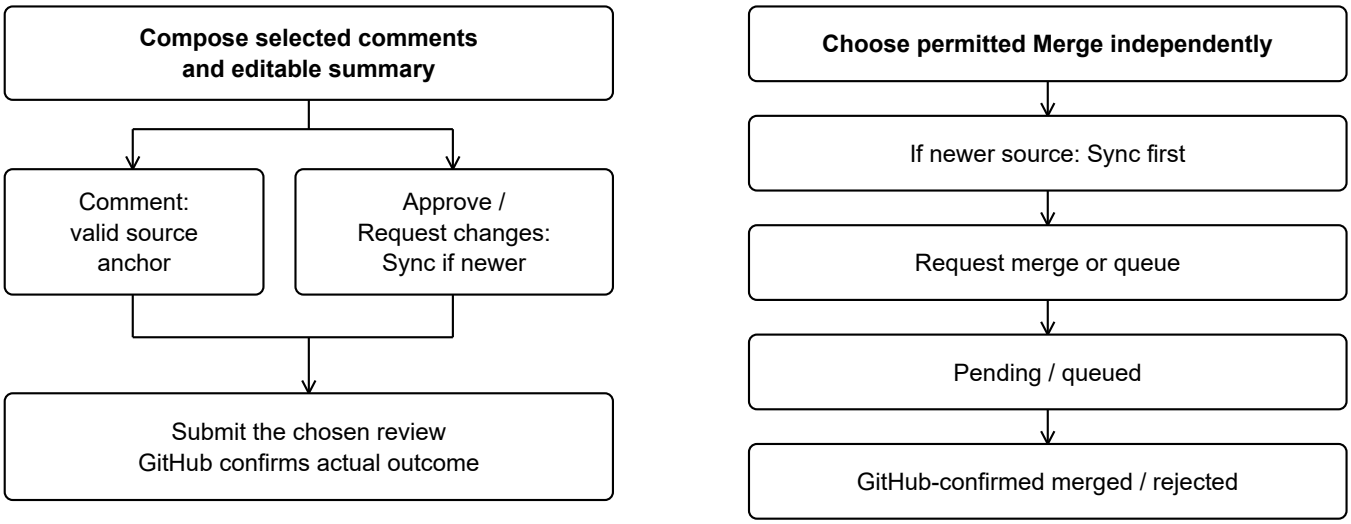


Figure 08. Review submission and Merge are independent actions. The Sync-first gate applies to Approve, Request changes and Merge; earlier-code comments can remain valid.

Submission rules

The ordinary submission composer shows selected comments, editable summary, chosen review decision and reviewed source version. It is the composition flow, not an additional blanket confirmation ceremony. Newer unsynced commits pause Approve, Request changes and Merge and offer Sync now; a context-specific Sync-first popup is allowed.

After Sync, affected areas remain flagged for rechecking. The workbench does not impose mandatory test execution or completion of every human Reviewed mark before a review decision. GitHub's own action rules remain binding; self-review does not grant an author self-approval where GitHub forbids it. S03

Merge and outcome rules

Show known eligibility, repository-required reviews/checks, conflicts and permission limitations; label eligibility unknown when it cannot be established. Offer enabled merge commit, squash and rebase methods, and required merge-queue submission when permitted. The signed-in operator may be an author or reviewer. Approval does not itself request merging. S04, S05

Outcome	Product interpretation
Published	GitHub has confirmed the comment or review.
Failed	Explain the known failure and retain recoverable input.
Unconfirmed	Check receipt before retrying; do not invent an outcome or duplicate feedback.
Pending / queued	A merge request is awaiting completion; it is not yet merged.
Merged / rejected	Reflect the state actually confirmed by GitHub.

Publication or merging does not implicitly Exit Review. Queue administration, bypass and cancellation are not additional MVP requirements.

11 / Verification and command interaction

Verification is optional and focused on a question the reviewer chooses to investigate. The agent can propose a command or run it in the terminal under normal permissions. Author-owned regression assurance and repository CI remain separate responsibilities.

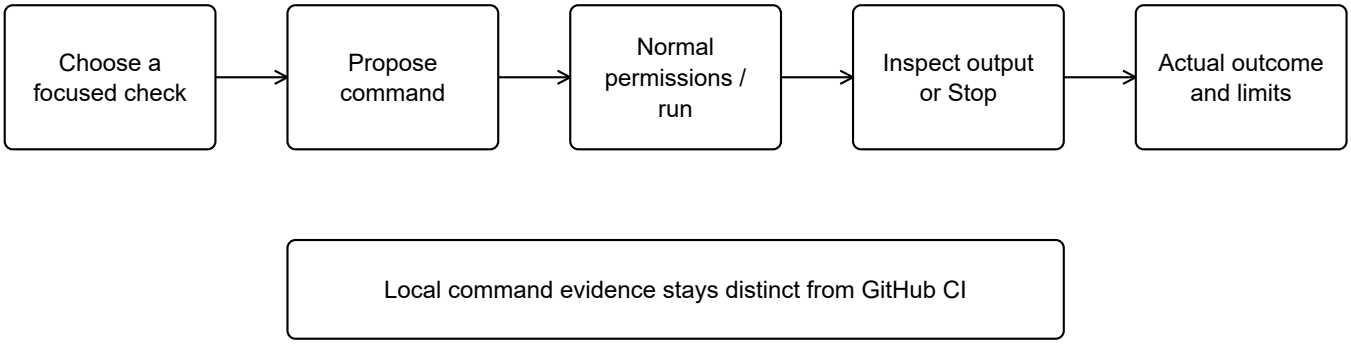


Figure 09. Verification supplies bounded evidence. It is not a required step or a judgment of review quality.

Familiar permissions and controls

Routine task-scoped checks proceed under Auto-style permissions, pausing when the normal policy needs reviewer approval. Commands and outputs are inspectable as conversation tool activity. Saved command permissions are inspectable and revocable, and the reviewer can stop workbench-owned work. This interaction follows the familiar coding-agent pattern; an execution framework is not selected. S06

Within the requested review task, a command that changes reviewed files follows normal permissions without an extra source-change approval rule. Ordinary generated output, such as logs, coverage and build files, is allowed. This does not create an autonomous fixing workflow or a dedicated PR editing feature.

Explain missing prerequisites and propose suitable setup commands. Their execution follows the normal permission policy; no blanket automatic installation is promised. Do not blindly repurpose a user terminal that is already running a command. Existing terminal output may support investigation, but no universal guarantee about all unrelated processes is inferred.

Evidence reporting

Report known errors, interruptions and unknown outcomes accurately. A successful process exit alone does not prove the code or tests are correct. Explain what was checked and the limits of that observation; passing results do not establish untested behavior. Distinguish local results from GitHub CI, and flag current-source applicability when the code changes.

No separate structured check ledger, universal zero-test classifier or verification dashboard is required. Active-PR source Sync waits for running workbench verification to finish or stop; discussion refresh remains independent.

12 / Profile, AI access and private data

Personal guidance

One editable personal review profile is associated with each GitHub account and initially saved on the current device. It acts as reusable instructions, such as focus areas or preferred explanation depth, across repositories. Relevant repository instructions are taken into account automatically.

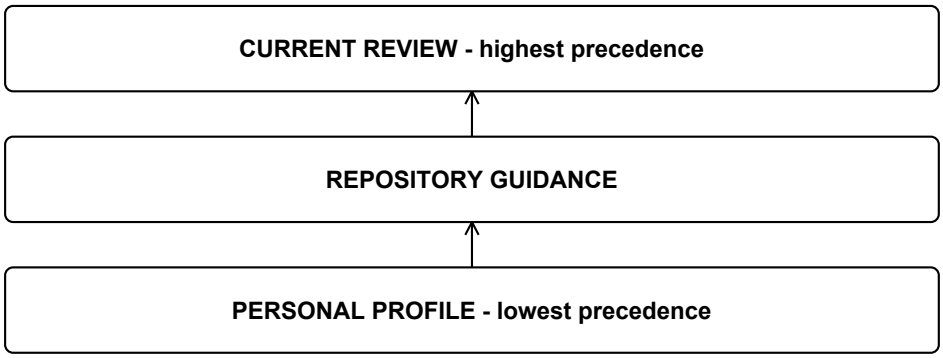


Figure 10. Guidance dominance increases from personal profile to repository to current-review instructions. This is a behavioral rule, not a prompt implementation.

Apply this hierarchy to ordinary guidance conflicts. The agent can explain the active guidance and precedence when asked; do not automatically flag every conflict. Preferences cannot change factual results or bypass permission and publication rules. Future style portability through Markdown export or equivalent is a direction, not an MVP feature.

AI and data boundaries

Item	Agreed MVP behavior
AI connection	Use the reviewer's own supported cloud-AI account or credentials. Claude, Gemini and ChatGPT are named examples; exact supported coverage/access remains open.
Provider choice	Switching to another provider requires deliberate user choice, including after failure.
AI unavailable	Keep manual local review and permitted GitHub collaboration usable when their own access remains available.
Private saved state	Save review conversation, context, progress, dismissals and unfinished local feedback on device by default initially.
Clear PR data	Clear that PR's private saved workbench state without deleting repository work, the account profile or GitHub records.

The selected cloud connection processes the context supplied to it. Local saved-state storage is separate from provider-side handling, and GitHub pending reviews remain GitHub records. Clearing local data does not promise deletion from external providers.

The proposed dedicated data-processing notice and file/folder exclusion controls are not MVP requirements. Local inference and a mandatory free assistant path are deferred/rejected for MVP. Account compatibility, inference costs, quotas and practical limits remain dependent on supported connections; every existing subscription is not automatically compatible.

13 / FR01-FR04: opening and orientation

FR01 - Open the selected change safely

Required behavior. Open a relevant GitHub PR through the repository-filtered personal list or a pasted link, or inspect local commits/branches and uncommitted changes. Support any-language/mixed-language entry and one active review. For a PR, obtain the local repository, preserve prior unfinished work and automatically enter the target repository/branch. Explicit Exit Review restores the previous workspace and work.

Observable checks. Missing local copies initiate cloning; failed setup offers recovery and does not claim remote-only review. A local read fallback leaves prior work intact when switching is unsafe. Open/history filters and relevant-list categories work. Local-only review has no invented GitHub destination.

FR02 - Explain intent and give a provisional assessment

Required behavior. Automatically show a concise plain-language explanation of the intended change, implementation and relevant uncertainty. Consider PR description and code together and mention material disparities. Material implementation claims link to applicable sources.

Observable checks. No separate author/inferred intent label scheme is required. Missing requirements are not fabricated. The opening take neither approves the PR nor certifies its correctness, and it does not initiate an exhaustive suggestion pass.

FR03 - Offer a flexible review route and explicit progress

Required behavior. Offer optional logical checkpoints while keeping every changed file directly available. Support agent-requested and drag/drop reordering with equivalent keyboard controls. Human Reviewed marks are explicitly set by the reviewer.

Observable checks. Ignoring or rearranging the route does not hide files. Reading code, an explanation or an AI suggestion does not mark human review completion. Sync clears marks on changed files, preserves unaffected marks and adjusts checkpoints only as needed to cover incoming changes.

FR04 - Connect explanations to ordinary source navigation

Required behavior. Open source references in normal VS Code tabs and use supported definition/reference navigation or existing workspace search. Opening code for review supplies a brief shared file-orientation summary explaining its role in the PR and current route.

Observable checks. References identify the source they describe. The wider PR/conversation remains primary context; cursor-by-cursor focus tracking is not required. Code stays central when the assistant is expanded, collapsed or adapted to a narrow window.

14 / FR05-FR08: conversation and comments

FR05 - Import published GitHub conversation

Required behavior. Show accessible published PR-level comments, submitted review summaries and inline threads/replies with authors, relevant time/source information and GitHub links. Preserve available resolved/outdated status and accessible conversation across additional result pages.

Observable checks. Other users' private pending drafts are not presented as published discussion. Partial access/loading is labeled rather than implying complete import. Restore the signed-in reviewer's own existing pending review through the separate draft flow.

FR06 - Refresh discussion and use attributed context

Required behavior. Refresh on entry, on demand and automatically during active review, independently of source-code Sync. Use relevant discussion to inform investigation and avoid duplicate suggestions.

Observable checks. New replies appear without requiring source checkout changes. Existing concerns link to their thread. Loading failures/incomplete scope remain visible. A collaborator's statement is attributed and does not become a verified code observation merely because it appears in discussion.

FR07 - Publish chosen comments and questions

Required behavior. Compose PR-wide, file-level or line/range feedback, including human questions and GitHub suggested-change blocks. Publish visible final text under the signed-in reviewer's identity through an accurate accepted destination. Follow immediate Comment and pending-review flows.

Observable checks. Assistant chat and unselected AI text are not automatically posted. Older-source comments retain accurate accepted anchors; invalid anchors never move silently to unrelated code. Publication failures preserve input and offer recovery. Local-only investigation does not invent a public destination.

FR08 - Reply and manage permitted thread actions

Required behavior. Reply within existing threads, edit/delete the reviewer's own comments and resolve/reopen threads where GitHub permits. Incorporate changes made by collaborators who continue entirely on GitHub.

Observable checks. Replies remain in the original discussion. Permission refusal explains the unavailable action and preserves recoverable private input. A workbench account is not required for collaborators. The agent does not autonomously resolve discussion or change GitHub's outdated state.

15 / FR09-FR12: suggestions, reviews and merges

FR09 - Offer and selectively publish AI suggestions

Required behavior. Generate reviewer-paced candidates on entering changed files. Use the agreed category/impact tags and default-collapsed explanations with evidence, reference links and verification limits. Permit impact/text adjustment, one-action dismissal and deliberate adoption for GitHub feedback.

Observable checks. Combine duplicate underlying concerns and link existing discussion. Retain same-PR dismissals unless meaningful new evidence or relevant changes justify resurfacing. Agent coverage is separate from human Reviewed marks. No mandatory question backlog or automatic publication is introduced.

FR10 - Submit an explicit GitHub review

Required behavior. Support Comment, Approve and Request changes, chosen by the reviewer. Restore unfinished feedback and the user's own pending review; permit discard. The normal composer shows selected comments, editable summary, chosen decision and reviewed version.

Observable checks. Newer unsynced commits pause Approve/Request changes and offer Sync. Drafts survive this transition. Only selected feedback is submitted; published content is not reposted. Tests and Reviewed-mark completion are not workbench-imposed prerequisites. GitHub governs self-approval and review validity.

FR11 - Request a separate permitted merge

Required behavior. Show known eligibility/blockers and offer repository-enabled merge commit, squash/rebase or required queue submission where the signed-in operator has authority. Merge is a separate action independently available under GitHub rules to an author or reviewer; workbench review submission is not its prerequisite.

Observable checks. Newer unsynced source pauses Merge. Do not bypass permissions, conflicts or required repository checks. Unknown eligibility remains unknown. Approval does not automatically request merging, and queue administration is outside the agreed capability.

FR12 - Report real publication and merge outcomes

Required behavior. Distinguish published, failed and unconfirmed delivery, and pending/queued merge requests from confirmed merged/rejected states. Preserve recoverable drafts/input on failure.

Observable checks. Check uncertain receipt before retrying to avoid duplicates. An accepted or queued request is never labeled merged without GitHub confirmation. Authentication/access/rate-limit failures provide recovery. Submitting or merging does not implicitly Exit Review or restore the prior workspace early.

16 / FR13-FR16: context and verification

FR13 - Investigate with relevant, inspectable context

Required behavior. Start from the current PR's changes and conversation, supplementing relevant guidance, discussion and attachments. Read/search supporting source when needed. Answer private questions and investigate concerns with inspectable sources and clear known limitations.

Observable checks. Support text/code/Markdown/PDF-text attachments. Unreadable or partial material is not represented as fully understood. No routine whole-repository ingestion or manual retained-source picker is required. Failed/stopped investigation remains visible; hypotheses and observed facts remain distinct.

FR14 - Retain same-PR understanding with freshness

Required behavior. Preserve relevant observations, decisions and context for continued investigation within the same PR. Flag affected or uncertain evidence when its supporting source changes. Retrieve another PR's context only through an explicit reference.

Observable checks. No proactive cross-PR relationship discovery or unprompted review-memory reuse. Unaffected progress survives updates. An older approval or check result does not become current evidence by retention. Sources are inspectable; dedicated exclusion/curation controls are not MVP requirements.

FR15 - Support focused terminal checks under normal permissions

Required behavior. Let the reviewer choose focused verification using existing workspace tools. The agent may propose or run commands with Auto-style permission behavior, inspectable conversation activity, Stop controls and inspectable/revocable saved grants.

Observable checks. Routine task-scoped commands proceed, pausing where normal permission requires approval. Source-changing commands use the same policy without an extra layer. Generated output is allowed. Do not blindly repurpose an occupied user terminal. Missing prerequisites are explained with setup proposals, not unrestricted automatic installation. Autonomous fixing remains outside the agreed workflow.

FR16 - Explain actual verification evidence

Required behavior. Make command/output activity inspectable and explain actual known outcomes and limitations. Separate local results from GitHub CI. Account for interruption, errors and uncertainty without claiming tested correctness from generic tool completion.

Observable checks. Active-source Sync waits for running workbench checks to finish or stop. Source changes flag affected evidence for rechecking. Untested behavior is not declared verified. A structured code-version/time ledger, universal zero-test classifier or new dashboard is not required.

17 / FR17-FR20: practice and workspace

FR17 - Apply one editable account-linked profile

Required behavior. Provide one local personal instruction profile per GitHub account, editable for review focus and explanation preferences. Apply relevant repository guidance. Dominance is personal profile < repository < current-review instructions.

Observable checks. Explain active guidance and precedence on request rather than flagging ordinary conflicts automatically. Personal instructions do not rewrite facts, hide changed files or bypass permission/publication rules. Multiple selectable profiles and automatic cross-device sync are not included; Markdown-style portability is a future direction.

FR18 - Resume and clear private review state

Required behavior. On return, automatically restore same-PR conversation/context, focus, reviewed-file progress, dismissals and unfinished local feedback. Initially store private workbench state on-device. Allow clearing that PR's saved private data.

Observable checks. Apply latest-source Sync and affected-state reconciliation; keep unaffected progress. Do not require a separate session-management task. Clearing local review data does not delete repository work, account guidance, published GitHub records or the separate GitHub pending review. No per-PR source-edit retention feature is added.

FR19 - Respect supported cloud-AI choice and fallback

Required behavior. Use reviewer-provided supported cloud-AI access and require deliberate choice before changing provider. Keep manual review and permitted GitHub collaboration usable when AI fails, if the repository and GitHub remain accessible.

Observable checks. Do not promise compatibility with every account/subscription or access to another assistant's private history. Local models and a free/local inference gate are outside MVP. Dedicated data-processing notices and sensitive-path exclusions are not required. Exact connection methods, quotas/costs and processing limits remain open.

FR20 - Preserve a familiar, accessible editor experience

Required behavior. Keep code central in ordinary VS Code tabs; reuse navigation/search/terminal and editor theme. Provide collapsible assistant views, keyboard access and narrow-window adaptation. Label Private, AI suggestion and Published states in text.

Observable checks. Collapsing the assistant does not exit review. An explanation or suggestion does not complete human review. The workspace is not replaced by a standalone editor. Normal requested commands remain governed by FR15; this requirement does not impose a separate blanket approval screen.

18 / Open questions and future direction

Explicit uncertainties

Open item	What remains to be established	Requirement boundary already settled
Cloud connections	Exact provider coverage and compatible account/login/API access; each connection's cost and quota behavior.	Reviewers bring supported access; no silent switch and no guaranteed free inference.
AI quality and capacity	Practical responsiveness, useful context size and behavior across languages/large PRs.	Any-language/mixed-language entry and reviewer-paced assistance remain required.
Source-view presentation	Exact way to inspect sources, file summaries, agent coverage and tool activity.	Inspectability, code centrality and explicit human progress remain required.
Setup and restoration	Detailed recovery interaction across local environments and incidental workspace states.	Local copy, prior-work preservation and actionable recovery remain required.
Context processing	Practical PDF-text handling, partial input and context-capacity limits.	Text-only PDF scope; do not pretend unavailable material was read.
Practical value	Whether the combined experience improves a particular reviewer's workflow.	Intended value is theoretical; the product does not score reviews or impose success metrics.

Future direction, without a commitment or schedule

The user introduced portable personal style through Markdown export or equivalent. Local-model connections, dedicated source-editing features and executable add-ons may be reconsidered later. Broader Git hosts, marketplaces and organization administration are separate expansion decisions, not implied by this MVP.

There is no approved plan for proactive cross-PR review memory. Future work must respect the current explicit-reference boundary unless the product scope is deliberately changed. A repository profile or standing guidance is distinct from carrying earlier PR-specific decisions into a new review.

Cross-cutting behavioral constraints

Preserve private input and explain failures rather than inventing completed actions. Keep source freshness and publication destinations accurate. GitHub visibility/permissions remain authoritative for collaboration; selected cloud providers determine compatible processing/access. Missing evidence or zero suggestions does not certify correctness or complete review coverage.

No architecture, framework selection, implementation plan, delivery schedule or formal success-metrics section is included in this edition.

19 / General market comparison

This is a selective category comparison using official documentation accessed on 11 October 2026. It records advertised capabilities, not a hands-on ranking, complete inventory or measured quality comparison.

Alternative category	Established capability	Workbench's intended emphasis
GitHub native review	Diffs, comments, suggested changes and formal review decisions. S02, S03	Connect private understanding and source investigation to deliberately selected collaboration.
GitHub review inside VS Code	PR lists, checkout, source navigation, comments, reviews and merging already exist in the editor. S01	Coordinate briefing, optional route, continuing investigation, human progress and freshness.
General coding assistants	IDE conversations, local-code investigation and command interaction; coding agents also offer review modes. S07, S08	Make understanding a selected change and completing its GitHub review the primary journey.
AI review products and guided surfaces	IDE findings, contextual chat/customization and guided review experiences already overlap. CodeRabbit Change Stack includes logical layers, summaries, history and GitHub review submission. S09, S10	Combine reviewer-paced assistance, same-PR decisions, local investigation and supported provider choice in a code-centered review flow.

Positioning

For developers who must understand and own their review decision, the workbench aims to connect orientation, code, accumulated same-PR context, evidence and GitHub collaboration in one familiar workspace. The theoretical distinction is this deliberate combination and its reviewer-controlled behavior.

Individual features overlap substantially with existing products. Guided review, AI chat, source navigation and editor-based collaboration are not unique inventions. A stronger combined experience has not been demonstrated, and the general comparison does not establish a competitive moat.

Evidence limits

Official documentation supports the presence of a capability; it does not establish usefulness, reliability or developer preference. This booklet does not reproduce vendor speed, bug-detection or adoption claims. A feature absent from this short table is not necessarily absent from a competitor. Pricing, editions and access can change, and exact workbench provider compatibility is still open.

20 / References and revision basis

Official reference sources

All external sources below were checked on 11 October 2026. Links support platform behavior and the general comparison, not the theoretical benefits on pages 23-24.

- **S01.** VS Code: Work with GitHub - established editor-based PR workflows.
- **S02.** GitHub: Commenting on a pull request - feedback locations, suggestions and immediate/pending flows.
- **S03.** GitHub: Reviewing proposed changes - review submission and platform constraints.
- **S04.** GitHub: Merging a pull request - merge methods and repository rules.
- **S05.** GitHub: Merging with a merge queue - queue requests versus eventual outcomes.
- **S06.** Claude Code: Configure permissions - familiar command permissions and saved grants.
- **S07.** Claude Code: Use in VS Code - IDE conversation, source references and command interaction.
- **S08.** OpenAI: Codex CLI - local investigation, tools and review modes.
- **S09.** CodeRabbit: VS Code overview - IDE review, findings and agentic chat.
- **S10.** CodeRabbit: Change Stack overview - guided review surface and review submission; published 13 May 2026.

Specification provenance

This edition uses the supplied draft PRD v0.2, the alignment handoff and the recorded product-alignment conversation. The full decision history remains in the companion decision log; the questionnaire is not reproduced here. Shared understanding was confirmed on 11 October 2026.

FR01-FR20 identifiers are retained, but their descriptions are updated to the agreed scope. Major corrections include language-agnostic/mixed-language entry, the personal repository-filtered list, automatic local checkout/restoration, explicit-reference-only other-PR context, incremental collapsed suggestions, cloud-first access and one local account profile.

Formal success metrics and the detailed CodeRabbit report are removed. The final assessment is theoretical. Diagram geometry is explanatory rather than a finished interface specification, and no implementation or schedule is selected.

21 / Theoretical strengths

What could be distinctive about the combination

The proposed strength is a review-specific continuing environment: the developer opens a change, builds understanding incrementally, inspects evidence and carries selected feedback into GitHub. The following are reasons the combination may be useful. They are not demonstrated advantages over every alternative.

Design emphasis	Theoretical benefit	Why it follows from the agreed behavior
Continuing same-PR understanding	Less need to repeat a question's background after navigating or taking a break.	The changed code, conversation, relevant decisions and source-backed context resume together.
Assistance at reviewer pace	Findings may stay closer to the area the reviewer is currently examining.	File entry triggers incremental investigation instead of an initial suggestion pass across the whole PR.
Human-owned review flow	Different review habits can remain useful within the same product.	Optional routes, adjustable impact, explicit human marks and selective publication avoid a mandatory AI queue.
Connected local evidence	Questions beyond the diff can be explored beside the code.	Existing navigation/search and optional terminal checks connect source inspection to the concern.
Coherent freshness handling	Updates can preserve useful work while making affected conclusions easier to revisit.	Sync retains unaffected progress and checkpoints while clearing changed-file marks and flagging affected evidence.
Personal guidance and supported AI choice	Review focus can reflect the user's and repository's needs without prescribing one provider forever.	One reusable profile, explicit instruction precedence and deliberate cloud-provider choice define the behavior.
GitHub-native collaboration destination	The reviewer can use the workbench without moving the rest of the team.	Published comments, review decisions and permitted merging remain ordinary GitHub interactions.

Where the claim stops

Existing review extensions, coding assistants and guided AI-review products already provide parts of this experience, sometimes in combination. The proposed distinction is the coherence of the full journey and these particular control/continuity choices. Its practical advantage, adoption burden and reliability remain uncertain.

The product offers assistance and truthful operational state. It does not guarantee exhaustive issue detection, eliminate human effort, assign review-quality scores or decide whether the reviewer invested enough time.

22 / Workflow friction and tradeoffs

Friction in a fragmented review workflow

These are plausible difficulties the product targets when a reviewer moves among GitHub, local code, assistant chat and terminal. They are not claims that every developer or existing tool suffers all of them.

Current-workflow friction	Workbench response
Repeating PR intent, source references and discussion in fresh chats.	Keep a continuing current-PR conversation and relevant inspectable context.
Moving between a diff, definitions, tests and private questions.	Keep code central and explanations connected to normal editor navigation.
Losing place after a break or author update.	Restore progress and reconcile affected material rather than restarting indiscriminately.
Repeating concerns already discussed.	Import current discussion and link overlapping suggestions to existing threads.
Manually transferring assistant feedback to GitHub.	Let the reviewer select, edit and publish exact feedback in the ordinary review flow.
Confusing a local check or accepted merge request with a stronger conclusion.	Distinguish local/CI evidence and pending/queued requests from confirmed outcomes.

Tradeoffs and cons of the proposed workflow

Constraint or cost	Practical consequence
Mandatory local repository and branch transition	Cloning, checkout, prerequisites and restoration add setup time and failure cases. Remote-only review is unavailable.
Incremental investigation	Unvisited areas may remain unexamined. A quiet suggestion list is not exhaustive assurance.
Saved contextual understanding	Retained statements can become stale or mistaken; provenance and rechecking remain essential.
Cloud AI and variable environments	Access, cost, responsiveness and model/tool behavior vary. Local inference is deferred.
One active review, GitHub.com-only MVP	Parallel review and broader hosting/team administration are outside the current scope.
User-controlled depth and publication	The reviewer still supplies judgment and decides how much evidence is enough; persuasive AI output can still be wrong.

The workbench intends to make the review experience coherent while keeping responsibility with the developer. Its own setup, context and integration burden must remain visible in the product description; theoretical convenience is not a demonstrated net advantage.